

Implementation and Verification of the Ricart-Agrawala Algorithm in Why3

Cláudia Faria (PG60240)
Patrícia Bastos (PG60287)

Universidade do Minho, Braga, Portugal
{PG60240, PG60287}@alunos.uminho.pt

June 19, 2026

1 Introduction

This project was done for the Formal Verification class in the Formal Methods of Programming specialization of the Master's in Informatics Engineering at the University of Minho.

The goal is to implement a concurrent algorithm in *Why3*, define properties that capture the correct behavior of that system and, finally, verify these properties.

2 The Ricart-Agrawala Algorithm

The chosen algorithm was the Ricart-Agrawala Algorithm, which was proposed by Glenn Ricart and Ashok Agrawala to be an extension and optimization of Lamport's Distributed Mutual Exclusion Algorithm. This algorithm was proposed to create mutual exclusion in a computer network whose nodes communicate only by messages and do not share memory.

Lamport's Distributed Mutual Exclusion Algorithm is a permission based algorithm proposed as an illustration of his synchronization scheme for distributed systems. [1] Ricart-Agrawala's Algorithm reduces complexity from $3(N - 1)$ to $2(N - 1)$ messages per critical section execution.

The full algorithm presented in the original paper is safe and live and claims to be an optimal solution. [2] It also includes mechanisms that exist to handle node insertion, removal and failure and introduces the possibility to reduce the

number of messages through modifications. These features will not be considered for this implementation.

The algorithm has no fault tolerance. This means that, in the case of a node failure, the algorithm will halt and the process will starve forever. This can be solved with failure detection by *timeout*. [3]

2.1 Algorithm

To Enter the Critical Section

When a site S_i wants to enter the critical section, it sends a timestamped **REQUEST** message to all other sites.

When a site S_j receives a **REQUEST** message from site S_i , it sends a **REPLY** message to S_i if and only if:

- Site S_j is neither requesting nor currently executing the critical section, or
- Site S_j is requesting, but the timestamp of S_i 's request is smaller than its own request.

To Execute the Critical Section

Site S_i enters the critical section if it has received a **REPLY** message from all other sites.

To Release the Critical Section

Upon exiting, site S_i sends **REPLY** messages to all deferred requests.

2.2 Characteristics

- The logical clock increments for every cycle.
- The states are **Requesting**, **Critical Section** and **Non Critical Section**.
- When a process wants to enter the critical section, it sends a timestamped **REQUEST** message to all other processes.
- Each process stores this **REQUEST** in its queue. It is a tuple (timestamp, PID).
- The queues determine who gets to use the critical section first.
- When a process exits the critical section, it sends a **REPLY** to all the deferred requests.

2.3 Assumptions

- Processes don't fail.
- Messages are never lost.
- Finite channel latencies (value is unknown).

2.4 Desired properties

Because *Why3* is not a temporal logic model checker, focus was given to safety properties. Based on the above algorithm characteristics, the following properties were derived.

- A process must receive REPLY message from all other processes before entering the critical section.
- A process can only send a REPLY message if it is neither requesting nor currently executing the critical section.
- If a process A and B both made a REQUEST, the process with the smallest timestamp will receive a REPLY message from the other process.

3 Solution

3.1 Implementation of the algorithm

The first step to implement the algorithm was modeling its components.

Type	Meaning
<code>pid</code>	Process identifier. Represents a unique process in the distributed system.
<code>timestamp</code>	Logical clock value used to order requests.
<code>st</code>	Process state. Can be <code>NCS</code> (Non-Critical Section), <code>Requesting</code> , or <code>CS</code> (Critical Section).
<code>request</code>	Structure containing a request timestamp (<code>rts</code>) and process identifier (<code>rp_id</code>).
<code>p_queue</code>	Queue of deferred requests, represented as a list of <code>request</code> .

Table 1: Types used in the Ricart–Agrawala formalization

The world was defined with all the necessary components for each state, with:

Field	Meaning
state	Maps each process identifier to its current state (NCS, Requesting, or CS).
n_replies	Stores the number of reply messages received by each process.
last_req	Stores the timestamp of the most recent request made by each process.
num	Logical clock value associated with each process.
queues	Stores the queue of deferred requests for each process.

Table 2: Global system state (**world**)

3.1.1 Initial state

All processes start outside the critical section with zero replies, empty queues and logical clocks set to 0.

3.1.2 System transitions

There are several system transitions modeled as predicates:

- **request_Send**: models the action of sending a request to enter the critical section. It is only enabled in **NCS**, changes state to **Requesting**, increments the clock and stores the timestamp.
- **enter_CS**: models the action of entering the critical section. It is only enabled inside **Requesting** and with $n - 1$ replies. Changes state to **CS**.
- **leave_CS**: models the action of leaving the critical section. It sends previously deferred replies, clears the queue and resets state, reply count and request timestamp.
- **request_Receive**: models the action of updating the queue and incrementing the clock upon receiving a new request.
- **reply_Send**: models the action of sending a reply upon reception of a request. It is immediate when a process is in **NCS** or **Requesting** with lower priority.
- **reply_Receive**: models the action of receiving a reply upon sending a request. It increments the count.

3.2 Invariants and safety properties

3.2.1 Inductiveness

To formalize mutual exclusion and full permission to enter the CS, we use an inductive invariant. This is a property that, if held in the initial state and after every transition, then it holds for every single state.

It is defined as:

For any two processes p and q , if both are in the critical section (CS), then they must actually be the same process.

and

For a total of n processes, and for any process p , if p is in the critical section (CS), then it must have received $n - 1$ replies.

In simpler terms, at most one process can be in the critical section at any time and if a process is in the critical section, it must have received replies from all other processes.

The next subsections will cover each individual goal addressed.

3.2.2 Mutual exclusion

The first goal seeks to verify if there cannot be two different processes in the critical section, using:

For processes p and q , and for any reachable world w , if both processes are in the critical section, then they are the same process.

This goal follows directly from the invariant and was successfully verified.

3.2.3 Collecting all replies

The second goal seeks to verify that a process cannot enter the critical section prematurely:

For any process p , and for any reachable world w , if p is in the critical section, then it has received replies from all other processes.

This goal follows directly from the invariant and was successfully verified.

3.2.4 Sending valid reply

The third goal seeks to verify that a process in the critical section cannot arbitrarily send a reply:

For any process p , any request r , and for any reachable world w , if a process is allowed to send a reply, then its state must be **NCS** or **Requesting**.

This goal follows from the definition of `reply_Send_enbl` and was successfully verified.

3.2.5 Fairness

The fourth goal seeks to verify fairness. That is,

For two different processes p and q , and for any reachable world w , if both processes are in **Requesting**, then the process with the smaller timestamp will not have the other process in its queue.

By not having a requesting process in its queue, the process has already received the reply.

Unlike the other goals, this cannot be proven directly from the invariant or definitions. The goal closed but, upon closer inspection, it was found to be closing vacuously. An experiment was done to test this by running the negation of the goal, which also closed.

This result hints that the current invariant and definitions are not strong enough to capture the complexity of this goal. This would be due to the fact that the model can't reason about the content of the queues of reachable states. This goal was dropped and left for future work.

3.3 Refinement

3.3.1 Mutex Specification

The **MutexSpec** module defines the simplest possible model of mutual exclusion by modeling just presence in the critical section. In this case, the **world** is defined just as mapping from PID to a boolean, indicating if a process is in the critical section or not. It defines an invariant for the abstract mutual exclusion property.

This module is important to define the abstract correctness target of the algorithm, modeling its most essential requirement: mutual exclusion. The refinement mapping will be done from the previously concrete implementation to this abstract specification.

3.3.2 Refinement mapping

The refinement mapping is a function that formally connects the concrete and abstract specifications of a system, by mapping states from one state machine to another. Here, the refinement mapping maps concrete states (from the Ricart-Agrawala module) to abstract states, losing the details that don't concern mutual exclusion.

In short, we seek to prove:

$$\text{Ricart-Agrawala} \sqsubseteq \text{MutexSpec}$$

This mapping is essential to the hierarchical verification to formally prove the correct implementation of the problem (in this case, mutual exclusion). In Why3, these mapping conditions can be incorporated into the contracts of

action functions as Refinement Contracts, which allows for the generation of verification conditions (VCs) to ensure that the mapping is valid during the execution of each action in the system.

3.3.3 Proof of refinement

The time has come to prove that the concrete system simulates the abstract one. The proof of refinement will check three things:

- the initial states must correspond;
- the concrete invariants apply the abstract invariants;
- each concrete step matches an abstract step.

This verification is done by checking equality of states, predicates and functions between specifications. As expected, the mutual exclusion property was verified.

4 Results, Limitations and Future Work

The invariant was designed to capture the essential safety conditions of the Ricart-Agrawala protocol: mutual exclusion and complete authorization before critical-section entry. However, the fourth goal, which sought to verify some sort of fairness, was dropped due to the invariant’s insufficient strength. To prove it, a stronger invariant concerning queue contents and deferred request consistency would be necessary.

While sufficient for proving safety, the current invariant is not strong enough to establish liveness or fairness properties. As stated earlier in this report, this is also a limitation of *Why3*. Using this tool, it is easier to directly model safety properties, and goals such as the fourth one are much more complex. However, recent research [4] adds temporal behavior and weakest-precondition reasoning over executions, which is currently outside *Why3*’s core model. This research is promising and could be helpful to prove more complex properties such as liveness and fairness.

The refinement provided an abstract view of essential protocol details and proved that the Ricart-Agrawala implementation faithfully simulates an ideal mutual exclusion specification, allowing safety properties to be inherited.

Future work would include trying to strengthen the invariant to produce more satisfactory results and explicitly proving the abstract system’s correctness.

5 Conclusion

In this project, the Ricart-Agrawala algorithm was implemented in *Why3* to prove its most important properties: mutual exclusion, validation of ability to send a reply, and full reception of all replies before entering the critical section.

Almost all properties were verified, with the exception of a liveness property that the invariant could not hold and inevitably fell out of scope of the tool's lack of temporal logic handling.

The algorithm was then mapped to an abstract specification that allowed it to focus on mutual exclusion and inherit verified safety properties.

The project leaves room for further verification of other properties using new techniques and more careful verification of the abstract specification in the refinement, as well as the addition of a stronger invariant.

References

- [1] *Lamport's Algorithm for Mutual Exclusion in Distributed System*, GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/operating-systems/lamports-algorithm-for-mutual-exclusion-in-distributed-system/>
- [2] G. Ricart and A. K. Agrawala, *An Optimal Algorithm for Mutual Exclusion in Computer Networks*, Communications of the ACM, vol. 24, no. 1, pp. 9–17, 1981. Available at: <https://dl.acm.org/doi/10.1145/358527.358537>
- [3] *Ricart-Agrawala Algorithm in Mutual Exclusion in Distributed System*, GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/operating-systems/ricart-agrawala-algorithm-in-mutual-exclusion-in-distributed-system/>
- [4] G. Ernst, *A Weakest Precondition Calculus for Programs and Linear Temporal Specifications*, arXiv preprint arXiv:2602.10746, 2026. Available at: <https://arxiv.org/abs/2602.10746>